

Part 1 – Rental Housing Web App

Code

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Assignment1-Farzaan</title>

  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      font-family: "Trebuchet MS", Verdana, sans-serif;
      background: #f3eee7;
      color: #292929;
    }

    .container {
      width: 92%;
      max-width: 480px;
      background: #fffdf9;
      padding: 29px 34px;
      border: 2px solid #6c8b83;
      border-radius: 9px;
      box-shadow: 7px 8px 0 #d8c8b8;
    }

    h1 {
      text-align: center;
      margin-top: 2px;
      margin-bottom: 25px;
      color: #514365;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Rental Housing</h1>
  </div>
</body>
</html>
```

```
        font-size: 27px;
    }

    label {
        display: block;
        margin-bottom: 6px;
        font-size: 15px;
        font-weight: bold;
    }

    input,
    textarea,
    select {
        width: 100%;
        padding: 9px 11px;
        margin-bottom: 16px;
        border: 1.5px solid #9c8eaa;
        border-radius: 5px;
        background: #fff;
        font-family: "Trebuchet MS", Verdana, sans-serif;
        font-size: 14px;
    }

    input:focus,
    textarea:focus,
    select:focus {
        outline: 2px solid #d98b73;
        border-color: #d98b73;
    }

    textarea {
        min-height: 90px;
        resize: vertical;
    }

    select {
        cursor: pointer;
    }

    .terms {
        display: flex;
        align-items: center;
```

```

        gap: 9px;
        margin: 3px 0 20px;
    }

    .terms input {
        width: 16px;
        height: 16px;
        margin: 0;
    }

    .terms label {
        margin: 0;
        font-weight: normal;
        font-size: 14px;
    }

    button {
        width: 100%;
        padding: 11px;
        background: #596f68;
        color: #fffd9;
        border: 2px solid #42564f;
        border-radius: 5px;
        font-family: "Trebuchet MS", Verdana, sans-serif;
        font-size: 15px;
        cursor: pointer;
    }

    button:hover {
        background: #485f57;
    }
</style>
</head>

<body>
    <div class="container">
        <h1>Rental Housing Listing</h1>

        <form id="rentalForm">
            <label for="propertyTitle">Property Name</label>
            <input
                type="text"

```

```
        id="propertyTitle"
        name="propertyTitle"
        placeholder="Enter the rental listing title"
        required
        autofocus
    >

<label for="location">Location</label>
<input
    type="text"
    id="location"
    name="location"
    placeholder="Enter the property location"
    required
>

<label for="submitterEmail">Email</label>
<input
    type="email"
    id="submitterEmail"
    name="submitterEmail"
    placeholder="Enter your email address"
    required
>

<label for="description">Property Details</label>
<textarea
    id="description"
    name="description"
    placeholder="Describe the rental property"
    required
></textarea>

<label for="propertyType">Property Type</label>
<select id="propertyType" name="propertyType">
    <option value="Apartment">Apartment</option>
    <option value="House">House</option>
    <option value="Condominium">Condominium</option>
    <option value="Townhouse">Townhouse</option>
</select>

<div class="terms">
```

```

        <input
            type="checkbox"
            id="termsAccepted"
            name="termsAccepted"
        >
        <label for="termsAccepted">
            I agree to the terms and conditions
        </label>
    </div>

    <button type="submit">Submit Listing</button>
</form>
</div>

<script src="script.js"></script>
</body>
</html>

```

Js code

```

const form = document.getElementById("rentalForm");

const makeCounter = () => {
    let count = 0;

    return () => {
        count++;
        return count;
    };
};

const countSubmit = makeCounter();

const validateForm = () => {
    const details = document.getElementById("description").value.trim();
    const terms = document.getElementById("termsAccepted").checked;

    if (details.length <= 25) {
        alert("The property description must be more than 25 characters.");
        return false;
    }
}

```

```
    if (!terms) {
        alert("Please agree to the terms and conditions.");
        return false;
    }

    return true;
};

form.addEventListener("submit", (e) => {
    e.preventDefault();

    if (!validateForm()) {
        return;
    }

    const data = {
        propertyTitle: document.getElementById("propertyTitle").value.trim(),
        location: document.getElementById("location").value.trim(),
        submitterEmail: document.getElementById("submitterEmail").value.trim(),
        description: document.getElementById("description").value.trim(),
        propertyType: document.getElementById("propertyType").value,
        termsAccepted: document.getElementById("termsAccepted").checked
    };

    const jsonData = JSON.stringify(data, null, 2);
    console.log(jsonData);

    const parsedData = JSON.parse(jsonData);
    const { propertyTitle, submitterEmail } = parsedData;

    console.log("Property title:", propertyTitle);
    console.log("Email:", submitterEmail);

    const updatedData = {
        ...parsedData,
        submissionDate: new Date().toISOString()
    };

    console.log("Updated listing:", updatedData);
    console.log("Submission count:", countSubmit());
});
```

Screenshots

Rental Housing Listing

Property Name
Farzaan Dalwai

Location
San Jose

Email
farzaan.dalwai@sjsu.edu

Property Details
3bhk pool clean spacious hot tub and pool built in ac

Property Type
House

☒ I agree to the terms and conditions

Submit Listing

Rental Housing Listing

Property Name: Enter the rental listing

Location: Enter the property location

Email: Enter your email address

Property Details: Describe the rental property

Property Type: Apartment

☐ I agree to the terms and conditions applied

Submit listing

Part 2:

```
import argparse
```

```

import json
import urllib.request

MODEL = "llama3.2:latest"

def call_ollama(prompt, temperature):
    data = {
        "model": MODEL,
        "prompt": prompt,
        "stream": False,
        "format": "json",
        "options": {
            "temperature": temperature
        }
    }

    req = urllib.request.Request(
        "http://localhost:11434/api/generate",
        data=json.dumps(data).encode(),
        headers={"Content-Type": "application/json"}
    )

    with urllib.request.urlopen(req) as response:
        result = json.loads(response.read().decode())

    return json.loads(result["response"])

def planner(title, content, temperature):
    prompt = f"""
You are a planner.

Title: {title}
Content: {content}

Create exactly 3 tags and a summary with no more than 25 words.

Return only JSON like this:
{{
    "tags": ["tag1", "tag2", "tag3"],
    "summary": "short summary"
}}

```

```

}}
"""

    return call_ollama(prompt, temperature)

def reviewer(title, content, plan, temperature):
    prompt = f"""
You are a reviewer.

Title: {title}
Content: {content}

Planner result:
{json.dumps(plan)}

Check that there are exactly 3 relevant tags and that the summary is accurate
and no longer than 25 words.

Return only JSON like this:
{{
    "tags": ["tag1", "tag2", "tag3"],
    "summary": "short summary",
    "changed": true,
    "reason": "short reason"
}}
"""

    return call_ollama(prompt, temperature)

def finalize(review):
    tags = review["tags"][:3]
    words = review["summary"].split()

    if len(words) > 25:
        words = words[:25]

    return {
        "tags": tags,
        "summary": " ".join(words)
    }

```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--title", required=True)
    parser.add_argument("--content", required=True)
    parser.add_argument("--temperature", type=float, default=0.0)

    args = parser.parse_args()

    plan = planner(args.title, args.content, args.temperature)

    print("\nPlanner output:")
    print(json.dumps(plan, indent=2))

    review = reviewer(
        args.title,
        args.content,
        plan,
        args.temperature
    )

    print("\nReviewer output:")
    print(json.dumps(review, indent=2))

    final = finalize(review)

    print("\nFinalized output:")
    print(json.dumps(final, indent=2))

if __name__ == "__main__":
    main()
```

SScreenshots:

```
Terminal
farzu@Farzaans-MacBook-Air data260-0686 % python3.11 agents_demo.py \
--title "Spacious 3 Bedroom House in San Jose" \
--content "Three bedroom rental house with a swimming pool, air conditioning, garage parking, and a large backyard near downtown San Jose." \
--temperature 0.0

Planner output:
{
  "tags": [
    "San Jose Rental",
    "Pool House",
    "Family Home"
  ],
  "summary": "Spacious 3 bedroom house with pool, AC, garage parking and large backyard near downtown San Jose."
}

Reviewer output:
{
  "tags": [
    "San Jose Rental",
    "Pool House",
    "Family Home"
  ],
  "summary": "Spacious 3 bedroom house with pool, AC, garage parking and large backyard near downtown San Jose.",
  "changed": false,
  "reason": ""
}

Finalized output:
{
  "tags": [
    "San Jose Rental",
    "Pool House",
    "Family Home"
  ],
  "summary": "Spacious 3 bedroom house with pool, AC, garage parking and large backyard near downtown San Jose."
}
```

Part 3:

```
import json
import csv
import time
from collections import Counter

from agents_demo import planner, reviewer, finalize

with open("reports/hw01/cases/nondeterminism_input.json") as file:
    test_input = json.load(file)

title = test_input["title"]
content = test_input["content"]

results = []

def percentile(values, percent):
    values = sorted(values)
    position = (len(values) - 1) * percent
    lower = int(position)
    upper = min(lower + 1, len(values) - 1)
    difference = position - lower
```

```

    return values[lower] + (values[upper] - values[lower]) * difference

def run_test(temperature):
    start = time.time()

    plan = planner(title, content, temperature)
    review = reviewer(title, content, plan, temperature)
    final = finalize(review)

    latency = (time.time() - start) * 1000

    return {
        "temperature": temperature,
        "tags": final["tags"],
        "summary": final["summary"],
        "latency_ms": round(latency, 2)
    }

for temperature in [0.7, 0.0]:
    print(f"\nRunning temperature {temperature}")

    for run_number in range(1, 21):
        result = run_test(temperature)
        result["run"] = run_number
        results.append(result)

    print(
        f"Run {run_number}/20 - "
        f"{result['latency_ms']} ms - "
        f"{result['tags']}"
    )

with open("reports/hw01/raw/part3_runs.json", "w") as file:
    json.dump(results, file, indent=2)

with open("reports/hw01/raw/part3_runs.csv", "w", newline="") as file:
    writer = csv.writer(file)

```

```

writer.writerow([
    "temperature",
    "run",
    "tags",
    "summary",
    "latency_ms"
])

for result in results:
    writer.writerow([
        result["temperature"],
        result["run"],
        " | ".join(result["tags"]),
        result["summary"],
        result["latency_ms"]
    ])

def calculate_metrics(temperature):
    temp_results = [
        result for result in results
        if result["temperature"] == temperature
    ]

    tag_sets = []
    tag_counter = Counter()
    latencies = []

    for result in temp_results:
        normalized_tags = [
            tag.strip().lower()
            for tag in result["tags"]
        ]

        tag_sets.append(tuple(sorted(normalized_tags)))
        tag_counter.update(set(normalized_tags))
        latencies.append(result["latency_ms"])

    distinct_sets = len(set(tag_sets))

    tags_all_runs = [

```

```

        tag for tag, count in tag_counter.items()
        if count == 20
    ]

    tags_one_run = [
        tag for tag, count in tag_counter.items()
        if count == 1
    ]

    return {
        "distinct_tag_sets": distinct_sets,
        "tags_all_runs": tags_all_runs,
        "tags_one_run": tags_one_run,
        "p50": round(percentile(latencies, 0.50), 2),
        "p95": round(percentile(latencies, 0.95), 2),
        "p99": round(percentile(latencies, 0.99), 2)
    }

metrics_07 = calculate_metrics(0.7)
metrics_00 = calculate_metrics(0.0)

print("\nPart 3 Results")

for temperature, metrics in [(0.7, metrics_07), (0.0, metrics_00)]:
    print(f"\nTemperature {temperature}")
    print("Distinct tag sets:", metrics["distinct_tag_sets"])
    print("Tags in all 20 runs:", metrics["tags_all_runs"])
    print("Tags in exactly 1 run:", metrics["tags_one_run"])
    print(
        "Latency p50 / p95 / p99:",
        metrics["p50"],
        "/",
        metrics["p95"],
        "/",
        metrics["p99"],
        "ms"
    )

with open("reports/hw01/METRICS.md", "w") as file:
    file.write("# Homework 1 Metrics\n\n")

```

```

file.write("| Metric | Temp 0.7 | Temp 0.0 |\n")
file.write("|---|---|---|\n")
file.write(
    f"| Distinct tag sets | {metrics_07['distinct_tag_sets']} | "
    f"{metrics_00['distinct_tag_sets']} |\n"
)
file.write(
    f"| Tags in all 20 runs | {'', '.join(metrics_07['tags_all_runs']) or 'None'} | "
    f"{'', '.join(metrics_00['tags_all_runs']) or 'None'} |\n"
)
file.write(
    f"| Tags in exactly 1 run | {'', '.join(metrics_07['tags_one_run']) or 'None'} | "
    f"{'', '.join(metrics_00['tags_one_run']) or 'None'} |\n"
)
file.write(
    f"| Latency p50 / p95 / p99 (ms) | "
    f"{metrics_07['p50']} / {metrics_07['p95']} / {metrics_07['p99']} | "
    f"{metrics_00['p50']} / {metrics_00['p95']} / {metrics_00['p99']} |\n"
)

print("\nFiles saved in reports/hw01/")

```

Screenshot

```

farzu@Farzaans-MacBook-Air data260-0686 % cat reports/hw01/METRICS.md
# Homework 1 Metrics

| Metric | Temp 0.7 | Temp 0.0 |
|---|---|---|
| Distinct tag sets | 15 | 1 |
| Tags in all 20 runs | None | san jose rental, family home, pool house |
| Tags in exactly 1 run | luxury rental, san jose housing, family home, city living, downtown san jose, pool included, poolhouse, sanjoserental, familyfriendly, swimming pool, house for rent, san jose ca, pool access | None |
| Latency p50 / p95 / p99 (ms) | 2953.05 / 3413.33 / 3542.47 | 2704.56 / 3005.8 / 3094.06 |
farzu@Farzaans-MacBook-Air data260-0686 %

```

```

● farzu@Farzaans-MacBook-Air data260-0686 % python3.11 part3_experiment.py

Running temperature 0.7
Run 1/20 - 2853.3 ms - ['San Jose Rental', 'Pool House', 'Family Friendly']
Run 2/20 - 2872.65 ms - ['Luxury Rental', 'San Jose Housing', 'Pool House']
Run 3/20 - 3288.71 ms - ['San Jose Rental', 'Pool House', 'Family Friendly']
Run 4/20 - 3401.48 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 5/20 - 2824.15 ms - ['San Jose Rental', 'Pool House', '3 Bedroom']
Run 6/20 - 2964.72 ms - ['San Jose Rental', 'Pool House', 'Downtown Area']
Run 7/20 - 2919.17 ms - ['San Jose Rental', 'Pool House', '3 Bedroom']
Run 8/20 - 2772.64 ms - ['San Jose Rental', 'Pool House', 'City Living']
Run 9/20 - 2941.37 ms - ['San Jose Rental', 'Pool House', '3 Bedroom']
Run 10/20 - 2645.64 ms - ['San Jose Rental', 'Pool House', 'Family Friendly']
Run 11/20 - 3484.83 ms - ['San Jose Rental', 'Swimming Pool House', 'Downtown San Jose']
Run 12/20 - 2978.28 ms - ['San Jose Rental', 'Pool Included', '3 Bedroom House']
Run 13/20 - 3225.1 ms - ['SanJoseRental', 'PoolHouse', 'FamilyFriendly']
Run 14/20 - 2696.32 ms - ['San Jose Rental', 'Swimming Pool House', 'Family Friendly']
Run 15/20 - 3193.64 ms - ['San Jose Rental', '3 Bedroom House', 'Downtown Living']
Run 16/20 - 2861.77 ms - ['San Jose Rental', 'Swimming Pool', 'Downtown Living']
Run 17/20 - 3034.2 ms - ['San Jose Rental', 'Pool House', 'Downtown Living']
Run 18/20 - 3574.76 ms - ['San Jose Rental', 'Pool House', 'Downtown Area']
Run 19/20 - 3185.76 ms - ['House for Rent', 'San Jose CA', 'Pool House']
Run 20/20 - 2854.21 ms - ['San Jose Rental', 'Pool Access', '3 Bedroom House']

Running temperature 0.0
Run 1/20 - 2914.37 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 2/20 - 2688.89 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 3/20 - 2707.63 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 4/20 - 2637.58 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 5/20 - 2675.51 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 6/20 - 2665.54 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 7/20 - 2686.88 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 8/20 - 2714.76 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 9/20 - 2999.99 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 10/20 - 2826.66 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 11/20 - 2631.89 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 12/20 - 2722.09 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 13/20 - 2659.9 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 14/20 - 2731.4 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 15/20 - 2759.1 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 16/20 - 2679.05 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 17/20 - 2701.48 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 18/20 - 2679.19 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 19/20 - 2757.02 ms - ['San Jose Rental', 'Pool House', 'Family Home']
Run 20/20 - 3116.13 ms - ['San Jose Rental', 'Pool House', 'Family Home']

Part 3 Results

Temperature 0.7
Distinct tag sets: 15
Tags in all 20 runs: []
Tags in exactly 1 run: ['luxury rental', 'san jose housing', 'family home', 'city living', 'downtown san jose', 'pool included', 'poolhouse', 'sanjoserental', 'familyfriendly', 'swimming pool', 'house for rent', 'san jose ca', 'pool access']
Latency p50 / p95 / p99: 2953.05 / 3413.33 / 3542.47 ms

```

Part4:

Model client

```

import json

import urllib.request

MODEL = "llama3.2:latest"
URL = "http://localhost:11434/api/chat"

def complete(messages, tools=None):
    data = {
        "model": MODEL,
        "messages": messages,
        "stream": False
    }

    if tools:
        data["tools"] = tools

    req = urllib.request.Request(
        URL,

```

```

        data=json.dumps(data).encode(),
        headers={"Content-Type": "application/json"}
    )

    with urllib.request.urlopen(req) as response:
        result = json.loads(response.read().decode())

    return {
        "message": result["message"]["content"],
        "input_tokens": result.get("prompt_eval_count", 0),
        "output_tokens": result.get("eval_count", 0)
    }

```

Client

```

import json
from src.model_client import complete

with open("AGENT.md") as file:
    system_prompt = file.read()

messages = [
    {
        "role": "system",
        "content": system_prompt
    }
]

turns = 0
total_input = 0
total_output = 0
token_data = []

def show_stats():
    print("\nStats")
    print("Turns:", turns)
    print("Input tokens:", total_input)
    print("Output tokens:", total_output)
    print("History length:", len(str(messages)))

```

```
print("Code Review Client")
print("Type /stats for statistics or /exit to quit.")

while True:
    user_input = input("\nYou: ")

    # Ignore blank input
    if not user_input.strip():
        continue

    if user_input == "/exit":
        break

    if user_input == "/stats":
        show_stats()
        continue

    messages.append({
        "role": "user",
        "content": user_input
    })

    result = complete(messages)

    messages.append({
        "role": "assistant",
        "content": result["message"]
    })

    turns += 1
    total_input += result["input_tokens"]
    total_output += result["output_tokens"]

    token_data.append({
        "turn": turns,
        "input_tokens": result["input_tokens"],
        "output_tokens": result["output_tokens"],
        "total_tokens": result["input_tokens"] + result["output_tokens"]
    })

    print("\nAssistant:")
    print(result["message"])
```

```

    print(
        "\nTokens - Input:",
        result["input_tokens"],
        "Output:",
        result["output_tokens"],
        "Total:",
        result["input_tokens"] + result["output_tokens"]
    )

print("\nFinal Stats")
show_stats()

with open("reports/hw01/raw/part4_tokens.json", "w") as file:
    json.dump(
        {
            "turns": token_data,
            "total_turns": turns,
            "cumulative_input_tokens": total_input,
            "cumulative_output_tokens": total_output
        },
        file,
        indent=2
    )

print("\nToken data saved to reports/hw01/raw/part4_tokens.json")

```

[AGENT.md](#)

You are a code reviewer.

Respond only with bullet points.

Keep the review short and clear.

Do not write paragraphs.

Focus on correctness, readability, and simple improvements.

Screenshots:

You: /stats

Stats

Turns: 3

Input tokens: 1516

Output tokens: 1107

History length: 5264

You: Can you review this code and tell me if anything should be improved? `def calculate_total(price, tax): return price + tax`

Assistant:

The provided code is a simple function that calculates the total cost by adding the price and tax. Here are some suggestions for improvement:

1. **Function naming**: The name 'calculate_total' could be more descriptive. Consider renaming it to something like 'add_price_and_tax' or 'calculate_final_cost'.
2. **Error handling**: The function assumes that the input price and tax will always be numbers. However, if non-numeric values are passed, a `TypeError` will occur. Consider adding error handling to ensure that the inputs can be converted to numbers.
3. **Type hinting**: Adding type hints for the function parameters and return value can improve code readability and help catch errors earlier.
4. **Input validation**: If tax is a percentage, it's usually represented as a decimal value (e.g., 0.08 for 8%). Consider adding input validation to ensure that tax is within a valid range.
5. **Separation of concerns**: The function calculates both the price and tax. Consider separating these into two separate functions to improve code organization and reusability.

You: /stats

Stats

Turns: 5

Input tokens: 4539

Output tokens: 2051

History length: 10100

You: /exit

Final Stats

Stats

Turns: 5

Input tokens: 4539

Output tokens: 2051

History length: 10100

Token data saved to reports/hw01/raw/part4_tokens.json